

第九章 演员-评论员算法

先回顾一下我们之前学过的内容：

1. 第四章和第五章我们学习了**基于策略的方法**：**策略梯度方法**及其优化。直接学习一个策略网络 θ ，输入状态 s ，直接输出采取各动作的概率 $\pi_\theta(a | s)$ 。
 - **优点**：在动作的选择这一方面能够有效适配连续动作的场景。连续动作时，网络一般会输出一个**动作的概率分布**，例如高斯分布 $a \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$ 。
 - **缺点**：在**动作的评价**这一方面，其依赖采集的轨迹得到的回报来评价；而同一个状态下采取同一个动作，最终回报可能因为环境随机性而不同，导致方差大，训练容易受到随机性影响。
2. 第六章到第八章我们学习了**基于价值的方法**：**DQN** 及其优化。直接学习一个网络来计算状态价值 $V_\pi(s)$ 以及动作价值 $Q_\pi(s, a)$ ，这样对动作的评价会更加稳定，方差小。其缺点是**难以应对连续动作场景**（虽然第八章介绍了针对连续动作的 DQN 方法，但是都不够好）。

于是我们有一个自然的想法：**结合两种方法之长**，提出演员（Actor）-评论员（Critic）算法。

其中，**Actor** 负责输出动作，使用策略梯度；**Critic** 负责评价状态或动作，使用 DQN。

优势演员-评论员算法 Advantage Actor-Critic A2C

用策略梯度负责更新策略，用价值函数和时序差分学习来提高评价稳定性。

Actor 和 Critic 都是神经网络，前者负责决定动作，后者负责判断这个动作好不好。

前者的参数记为 θ ，后者的参数记为 w 。

全流程：

1. 根据当前状态 s_t ，Actor **输出动作分布** $\pi_\theta(a | s_t)$ ，并从中选择一个动作 a_t 来执行。注意此处不一定选择概率最大的动作，而是可以保留一些**探索机制**。
2. 根据执行的动作 a_t ，环境返回这一步的经验 (s_t, a_t, r_t, s_{t+1}) 。一般来说，会收集多条这样的经验，然后统一进行后面的计算，平均后用于优化。
3. Critic 计算当前状态和下一状态的价值 $V_w(s_t)$ 以及 $V_w(s_{t+1})$ 。
4. **构造 TD 目标**： $y_t = r_t + \gamma(1 - d_t)V_w(s_{t+1})$ 。它由当前立即奖励和下一状态折扣价值组成，其中 d_t 表征“ s_t 是否为最终状态”：如果是则为 1，表明没有下一状态的价值部分，反之亦然。
5. 计算 TD 误差近似优势：**TD 误差**定义为 $\delta_t = y_t - V_w(s_t) = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$ ，可以近似为优势 $\hat{A}_t = \delta_t$ 。

6. **更新 Critic**: 预测应当接近 TD 目标, 故**构造损失函数**:

$$L_{\text{critic}} = \frac{1}{2} \left(y_t - V_w(s_t) \right)^2$$

通过梯度下降更新参数 w 。

7. **更新 Actor**: 参考策略梯度中的梯度:

$$\nabla_{\theta} J(\theta) \approx \sum_t A(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

上式是梯度加多经验平均写法, 转化为损失函数要加负号。可以将 Actor 的损失函数写为:

$$L_{\text{actor}} = -\hat{A}_t \log \pi_{\theta}(a_t | s_t)$$

通过梯度下降更新参数 θ 。

8. 全部完毕后, 结束本回合, 开启下一回合, 重复上述操作。

异步优势演员-评论员算法 A3C

简单来讲, 就是**派遣多个 A2C Worker 去收集经验**, 判断优化方向, 然后**异步优化全局参数**。

A3C 中有一个**全局网络** θ_{global} , 包含全局 Actor 和 Critic; 同时每个子 Worker 也有一套自己的**本地网络副本** $\theta_1, \theta_2, \dots, \theta_N$ 。

全流程:

1. 回合开始时, 每个子 Worker 将自己的本地网络副本与全局网络**同步**, 即 $\theta_i \leftarrow \theta_{\text{global}}$ 。
2. 每个子 Worker 出去工作, 流程和上面讲到的 A2C 完全一致, 从收集经验、构造目标和误差, 到计算梯度。当然, 所有环节都是根据本地网络, 也就是一开始的全局网络进行的。
3. **Worker 将本地计算出的梯度发送给全局网络, 用它更新 θ_{global} , 然后同步一份最新的 θ_{global} 到自己本地, 继续完成下一轮任务。**

注意 1: 不同的子 Worker 会在**不同的环境副本、不同的初始状态下启动**, 防止多个 Worker 走出一样的轨迹、采集一样的数据, 导致多开、收集更多数据的目的无法实现。

注意 2: 每个 Worker 的速度不同, 各 Worker 之间**互不干扰、不等待** (不会出现快的 Worker 等慢的 Worker 的情况)。这会导致**“过时梯度”**: 某个 Worker 发送给全局网络的梯度是基于初始状态开始的优化方案, 而此时全局参数可能已经被其他 Worker 优化过一次。这会**削减更新的稳定性和一致性**, 但在全局参数没有发生过大变化的情况下**可以接受**。

对比 A2C 和 A3C: **A2C 追求“同步、一致、稳定”; A3C 追求“异步、少等待、高吞吐”。**

注意: A3C 不一定优于 A2C, 这取决于硬件 (例如 CPU、GPU 等)。

路径衍生策略梯度

A2C 中 Critic 只告诉 Actor 当前动作好不好，而路径衍生策略梯度中还会告诉“应该往哪边调”。

与 A2C 的区别：路径衍生策略梯度中，Actor 通常输出一个**确定性动作** $\mu(s)$ ，Critic 输入 (s, a) ，输出 $Q(s, a)$ 。

此外，采取**目标网络**以及**经验回放池**的思想：

- **当前 Actor**： $\mu_{\theta}(s)$ ，输入状态，输出连续动作；
- **当前 Critic**： $Q_{\phi}(s, a)$ ，输入状态和动作，输出动作价值；
- **目标 Actor**： $\mu_{\bar{\theta}}(s)$ ；
- **目标 Critic**： $Q_{\bar{\phi}}(s, a)$ ；
- **经验回放池**：保存环境产生的 $(s_t, a_t, r_t, s_{t+1}, \text{done})$ 。

全流程：

1. 开始时，**目标网络通常复制当前网络参数**： $\bar{\theta} \leftarrow \theta$ ， $\bar{\phi} \leftarrow \phi$ 。
2. **当前 Actor 与环境交互**：Actor 根据当前状态 s_t 生成确定性动作 $\mu_{\theta}(s_t)$ ；为了探索，会加入**随机噪声扰动得到真正输出的动作**： $a_t = \mu_{\theta}(s_t) + \epsilon_t$ 。
3. 对环境执行这个动作，得到一条经验 $(s_t, a_t, r_t, s_{t+1}, \text{done})$ ，存入经验回放池。
4. **先用经验训练 Critic**：从经验回放池中随机采集一批经验，进行以下处理：
 - 先用目标 Actor 输入下一状态 s_{t+1} ，输出动作 $a_{t+1}' = \mu_{\bar{\theta}}(s_{t+1})$ ；
 - 再让目标 Critic 评价该动作： $Q_{\bar{\phi}}(s_{t+1}, a_{t+1}')$ ；
 - 得到目标值 $y_t = r_t + \gamma(1 - \text{done})Q_{\bar{\phi}}(s_{t+1}, \mu_{\bar{\theta}}(s_{t+1}))$ ，其中 done 为“是否最终状态”标记；
 - 当前预测为 $Q_{\phi}(s_t, a_t)$ ，于是得到损失 $L_{\text{critic}} = [Q_{\phi}(s_t, a_t) - y_t]^2$ ；
 - 通过梯度下降更新当前 Critic 参数 ϕ 。

本质上，环境告诉 Critic：这个状态、这个动作后来得到了什么结果；Critic 根据大量经验学会估计动作的长期回报。

5. **用 Critic 指导 Actor**：当 Critic 训练得比较可靠时，开始更新 Actor：
 - 首先，当前 Actor 对状态 s_t 输出一个动作 $a_t^{\text{actor}} = \mu_{\theta}(s_t)$ ；
 - 然后将 $(s_t, a_t^{\text{actor}})$ 送入当前 Critic，得到 $Q_{\phi}(s_t, \mu_{\theta}(s_t))$ ；
 - Actor 希望这个 Q 尽可能大，所以构造损失 $L_{\text{actor}} = -Q_{\phi}(s_t, \mu_{\theta}(s_t))$ ；
 - 这样形成了一条完整的前向传播路径： $\theta \rightarrow \mu_{\theta}(s_t) \rightarrow a_t^{\text{actor}} \rightarrow Q_{\phi}(s_t, a_t^{\text{actor}})$ 。

简单一点，就是 $\theta \rightarrow a \rightarrow Q$ 。现在我们需要反向传播更新 θ ，也就是 $Q \rightarrow a \rightarrow \theta$ 。

6. 当 Critic 和 Actor 都更新完毕后，更新目标网络，例如直接把当前网络复制给目标网络。

整个过程可以概括为：当前 Actor 与环境交互得到经验数据 → 根据经验，结合目标 Actor 和目标 Critic 训练当前 Critic → 用当前 Critic 指导当前 Actor 改进 → 完成一轮，当前网络和目标网络同步，继续下一轮。